# 采用 Cuda 对 BMP 图片进行卷积池化

## 1. 实验目的

本次实验的目的是利用 NVIDIA CUDA 异构计算平台，通过计算密集型的 BMP 图像处理任务，掌握 GPU 大规模并行编程模型与性能优化技术，具体包含以下核心内容：

a) **掌握 Cuda 并行编程：**深入理解 GPU 的异构计算架构(Host 与 Device)，掌握 CUDA 线程层次结构（Grid, Block, Thread），熟练使用 CUDA C++ 编写 Kernel 核函数，并实现 Host 端与 Device 端的高效内存管理与数据传输。

b) **实现复杂的并行算法：**针对 BMP 图像高斯卷积及 2 × 2 最大池化任务，利用 GPU 的 SIMT（单指令多线程）特性，设计并实现并行卷积与池化算法。通过建立每个线程与像素点的映射关系，实现百万级像素的同时处理，替代传统的串行循环。

c) **掌握关键并行策略与优化技术：**

- 基础实现：实现卷积与池化分步执行的基础版本，理解 Global Memory 的访问模式。
- 算法级优化探索：尝试利用高斯核的线性可分性，将 2D 卷积拆解为两次 1D 卷积，旨在降低计算复杂度。分析其在 GPU 架构下因引入额外显存读写（中间 Buffer）而导致的带宽瓶颈问题。
- 系统级优化实践：针对显存带宽瓶颈，实现“边卷积边池化”的算子融合技术。通过寄存器缓存中间结果，消除卷积与池化之间的显存 I/O 开销，构建高效的单 Kernel 流水线，以实现性能的极致提升。

d) **验证并行计算的正确性：**确保并行计算输出的图像文件，能通过验证程序与标准算法结果进行比对，在保证 MSE 极低且相似度达到 99% 以上的前提下，追求最高的处理速度。

e) **分析并行性能指标：**通过 cudaEvent 计时器精确测量 Kernel 执行时间（计算时间）与端到端运行时间（含数据传输），量化评估 GPU 相比

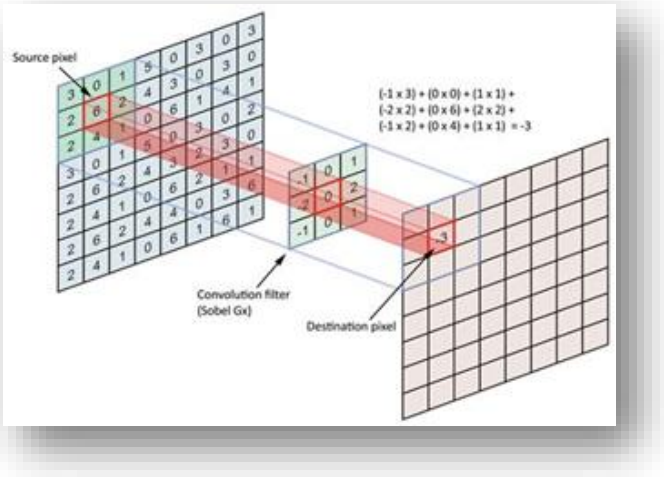
CPU 的加速比。对比“基础版”、“可分离版”与“融合版”的性能数据，分析计算密集型与访存密集型任务在 GPU 上的不同表现，理解显存带宽对小卷积核任务的决定性作用。

## 2. 并行策略与算法设计（基础版）

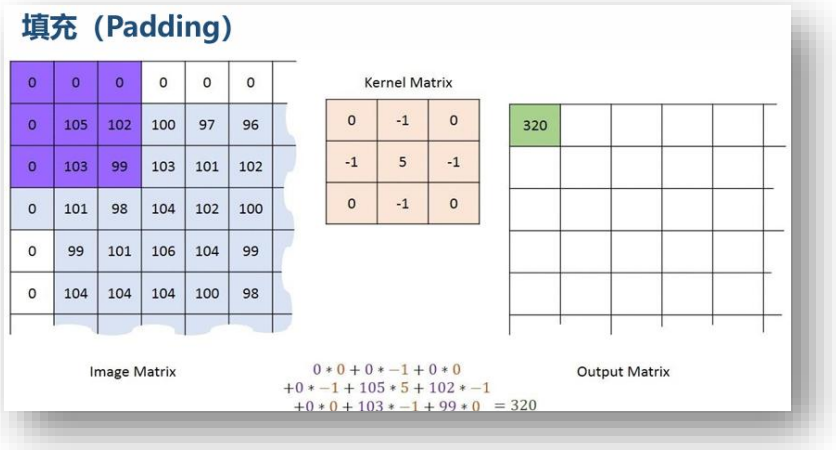
### a) 算法原理与设计

基础版并行算法基于 CUDA 异构计算模型，将传统的串行图像处理流程拆解为 Host（CPU）控制与 Device（GPU）计算两部分。针对实验要求的卷积与池化任务，采用分步执行的策略

- **双 Kernel 串行执行：**
  - **卷积阶段：**启动 convolution_kernel，计算源图像的 5×5 高斯卷积，将结果写入显存中的中间缓冲区。
  - **池化阶段：**启动 max_pool_kernel，从中间缓冲区读取数据，执行 2×2 最大池化，输出到最终目标内存。
- **数值处理：**卷积核权重存储于 GPU 的常量内存中以利用缓存机制，所有中间计算采用 float 精度，仅在最终写入时进行截断（0-255）和类型转换。




- **边界处理策略：**由于卷积窗口在处理图像边缘时会超出实际物理边界，本实验采用零填充策略。即当访问坐标超出图像范围时，默认该位置的像素值为 0，保证了卷积操作在全图范围内的统一性，且维持输出图像尺寸不变




b）并行化策略

为了充分利用 GPU 的海量 CUDA Core，本实验摒弃了 CPU 上的“按行分块”策略，采用了细粒度的二维网格映射策略

- **SIMT 线程映射模型：**实验利用 CUDA 的 SIMT（单指令多线程）模型，构建了 Grid-Block-Thread 的三级执行层次，定义了二维的线程块（Block），尺寸为 16×16，此配置确保了每个 Block 内的线程数未超过硬件限制（1024），且是 Warp Size (32) 的整数倍，有利于硬件调度器以 Warp 为单位高效发射指令，最大化指令吞吐率。

  网格（Grid）尺寸根据图像宽高动态计算，公式为 (Width+15)/16， 每个 GPU 线程通过内置变量计算唯一的全局坐标 (x, y)：

$$x = blockIdx.x \times blockDim.x + threadIdx.x$$

$$y = blockIdx.y \times blockDim.y + threadIdx.y$$

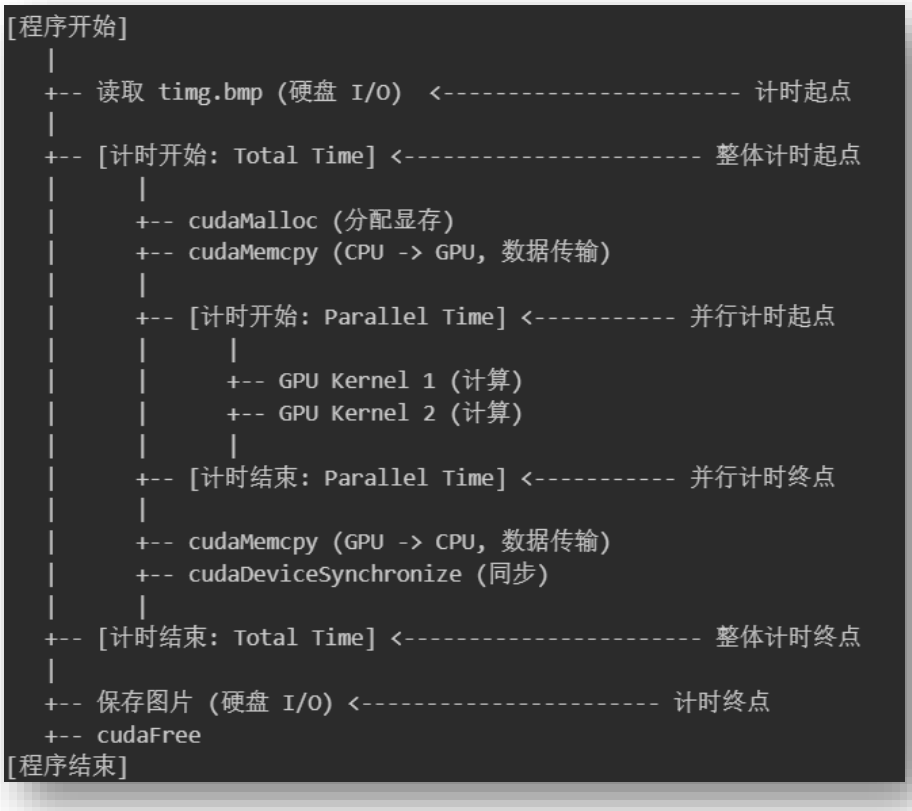
  每个线程负责**一个像素点**的计算任务，若坐标 (x, y) 超出图像边界，线程直接返回，实现自动的边界保护

- **内存层次与数据流：**
  - **Global Memory（全局内存）：**用于存储输入图像 d_src、中间卷积结果 d_conv 以及最终输出 d_pool，这是 GPU 容量最大但延迟最高的内存区域。

- Constant Memory（常量内存）：将 5×5 高斯卷积核 d_KERNEL 声明为 __constant__，由于所有线程读取相同的卷积核权重，常量缓存能提供极高的广播式读取带宽，显著优于全局内存读取。

- **显存带宽瓶颈（基础版痛点）**：在基础版设计中，卷积和池化被物理隔离为两个 Kernel，这意味着数据流必须经历：读原图 -> 算卷积 -> 写中间结果 -> 读中间结果 -> 算池化 -> 写最终结果。中间结果的写入与再次读取产生了大量冗余的显存 I/O，这通常是 GPU 算法的性能瓶颈所在。

c）流程图




1. 第一阶段：初始化与 IO，读取文件，计算参数
2. 第二阶段：资源分配与传输 cudaMalloc 分配显存，数据传输 cudaMemcpy，cudaMemcpyToSymbol 搬运到 gpu 上，配置 Kernel 启动参数
3. 第三阶段：并行计算核心，启动 convolution_kernel、max_pool_kernel，存储结果，等待 cpu 完成
4. 第四阶段：结果回传与清理，cudaMemcpy 传回 cpu，cudaDeviceSynchronize 同步确保操作结束，最后 cpu 释放资源

## 3. 核心代码分析

### a）数据结构与显存管理

为了适应 GPU 的异构内存架构，代码在数据结构定义与显存管理上进行了专门设计，以优化访存效率：

- **对齐的像素结构：**定义 struct Pixel { unsigned char b, g, r; }，利用 #pragma pack(push, 1) 确保内存紧凑排列。这不仅符合 BMP 24 位色深标准，也有利于 GPU 线程进行连续的内存读取。
- **常量内存优化：**考虑到卷积核权重在所有计算线程中是只读且共享的，本实验利用 CUDA 的 __constant__ 关键字定义卷积核

```
__constant__ float d_KERNEL[25];
// Host 端通过 cudaMemcpyToSymbol 初始化
CHECK(cudaMemcpyToSymbol(d_KERNEL, h_KERNEL, 25 * sizeof(float)));
```

相比于普通的 Global Memory，常量内存拥有片上缓存，当一个 Warp 内的 32 个线程同时访问同一个常量地址（如读取卷积核的同一个权重）时，硬件会触发广播机制，仅需一次内存事务即可满足所有请求，显著降低了密集卷积运算中的访存延迟。

### b）并行卷积核心逻辑

该 Kernel 是并行计算的核心，采用单线程处理单像素的细粒度并行策略，替代了 CPU 上的循环嵌套

- **SIMT 线程坐标映射：**摒弃了 CPU 的行分解策略，GPU 线程通过内置变量计算全局唯一的二维坐标 (x, y)，实现逻辑坐标与物理像素的一一对应

```
 int x = blockIdx.x * blockDim.x + threadIdx.x;
 int y = blockIdx.y * blockDim.y + threadIdx.y;

if (x >= out_width || y >= (in_height / 2)) return;
```

**自动边界防护：** 由于 Grid 尺寸通常向上取整以适配 Block 大小（16 的倍数），部分线程的坐标会超出图像范围，代码采用“早退”策略进行保护，避免非法显存访问

- **全局内存访问模式：**线程读取 d_src（输入图像）并写入 d_conv（中间结果），在卷积的 5×5 循环中，尽管 d_src 的访问具有一定的空间局部性，但由于未利用 Shared Memory，每次像素读取仍需访问高延迟的 Global Memory。

```
if (ny >= 0 && ny < height && nx >= 0 && nx < width) {
            Pixel p = input[ny * width + nx];
            float w = d_KERNEL[(ky + K_RADIUS) * 5 + (kx + K_RADIUS)];
            sum_r += p.r * w;
            sum_g += p.g * w;
            sum_b += p.b * w;
        }
```

**c）并行池化核心逻辑**

池化操作作为第二阶段的 Kernel 独立启动，体现了 CUDA 程序中 Kernel 间的数据依赖与同步机制

- **Grid 维度重映射：**由于池化会导致图像尺寸减半，启动该 Kernel 时重新计算了 Grid 大小，确保线程数与输出图像（W/2 × H/2）相匹配，而非原图大小

```
// 配置 Grid 和 Block
    dim3 block(16, 16);
    dim3 grid_conv((width + block.x - 1) / block.x, (height + block.y - 1) / block.y);
    dim3 grid_pool((out_width + block.x - 1) / block.x, (out_height + block.y - 1) / block.y);
```

**中间数据流转：**该 Kernel 读取上一阶段生成的中间显存 d_conv，执行 2×2 比较后写入最终显存 d_pool。

**瓶颈预判：**这种分步走的策略虽然逻辑清晰，但导致 d_conv 这一中间数据必须完整地写入显存再读出显存，在 GPU 架构中，Global Memory 的带宽通常是性能瓶颈，这种频繁的中间数据 I/O 为后续的算子融合优化留下了空间。

## 4. 实验结果分析

### a) 正确性检验

```
wyj2350489@ps:~/PThread/测试程序$ python3 test_pool.py timg.bmp Cuda_卷积池化_2350489.bmp
[Good] Image shape matched.
MSE result of the two pictures: 0.58
Similarity rate: 99.70%
```

```
----------------------------------------
1. 并行部分时间 (GPU Kernel) : 0.223232 ms
2. 整体运行时间 (H2D+Comp+D2H): 89.217 ms
3. 端到端时间 (含硬盘 IO)    : 210.806 ms
========================================
```

### b) 性能测试数据

| 并行部分时间（ms） | 整体运行时间（ms） | 端到端运行时间（ms） |
|---|---|---|
| 0.2232 | 89.217 | 210.806 |
| 0.2293 | 44.551 | 169.884 |
| 0.2371 | 59.946 | 181.97 |

### c) 结果分析

**1. 计算性能分析：**

核心计算时间稳定在 0.23ms 左右，相比于 CPU 串行或多线程版本，GPU 利用数千个 CUDA Core 实现了三个数量级的计算加速。

**2. 系统波动：**

整体运行时间在 44ms 至 89ms 之间剧烈波动，这是由于 cudaMalloc 显存分配、驱动初始化以及操作系统对 PCIe 总线的调度开销远大于实际计算时间，导致整体耗时受环境影响较大。

### 3．瓶颈分析

计算占比极低，GPU 仅花费了不到 0.4% 的时间在执行卷积和池化运算，而超过 99.6% 的时间消耗在了 Host 与 Device 之间的数据传输 cudaMemcpy 以及显存管理 cudaMalloc 上。

基础版算法需要将原图传入 GPU，计算出中间结果（卷积图），再读取中间结果计算池化，虽然计算瞬间完成，但数据的搬运受限于 PCIe 总线带宽和 Global Memory 的访问延迟。

基础版采用分步走策略（Conv Kernel + Pool Kernel），每次启动 Kernel 都有微秒级的 CPU-GPU 通信开销，对于 0.2ms 的超短任务，启动双 Kernel 的固有开销变得不可忽视。

### 4．优化思路引出

**算法复杂度优化（尝试可分离卷积）：**试图通过数学变换减少计算量，观察在 GPU 架构下，计算量的减少是否能抵消因引入额外中间显存带来的开销。

**访存模式优化（尝试算子融合）：**针对当前最大的瓶颈显存 I/O，尝试将卷积与池化融合为单一 Kernel，以计算换显存的策略，消除中间数据的读写，追求极致的访存效率。

## 5. 优化策略与算法改进

基于上一章节的性能分析，基础版并行程序的计算时间占比极低(<0.4%)，系统的主要瓶颈在于 Global Memory 的显存带宽，为了突破这一瓶颈，本实验先后探索了两种截然不同的优化路径：从降低计算复杂度的“可分离卷积”，转向降低访存开销的“算子融合”。

a) **优化策略：可分离卷积及其局限性**

首先尝试了算法层面的优化，利用高斯核的线性可分性，将 5 × 5 的二维卷积拆解为水平与垂直两次 1 × 5 的一维卷积

- 理论优势：该策略将每个像素的乘加运算次数从 25 次降低至 10 次，理论计算复杂度降低了 60%。

- 实际局限：在 GPU 架构下，计算量的减少未能转化为端到端的加速，原因是该算法引入了数据依赖性，必须分两步执行（行卷积 -> 列卷积），迫使系统增加了一个额外的中间显存缓冲区。
- 结论：对于这种小卷积核，GPU 的计算能力是过剩的，而显存带宽是稀缺的，可分离卷积虽然省了计算，却增加了显存读写，反而加剧了带宽竞争。因此，优化方向必须减少计转向减少访存。

b) **优化策略：算子融合**

针对带宽瓶颈，本实验最终采用了“算子融合”与“寄存器缓存”相结合的系统级优化策略，实现了边卷积边池化。

- **输出驱动的线程映射：** 摒弃了基础版“先算完所有卷积图，再算池化”的逻辑，改为直接以最终输出图像（池化后）的尺寸构建 Grid，每个 GPU 线程不再负责一个原图像素，而是负责生成一个最终的池化像素。
- **寄存器级数据流：** 由于 2 × 2 池化需要覆盖原图的 4 个像素，融合 Kernel 在单个线程内就地计算这 4 个位置的卷积结果。中间的卷积结果不再写入 Global Memory，而是直接保存在 SM（流多处理器）的寄存器中，线程在寄存器中直接对比这 4 个卷积结果，提取最大值，通过这种方式，彻底消除了卷积输出这一中间特征图的显存分配与读写。虽然单个线程的计算负载增加了 4 倍，但完全避免了高延迟的 Global Memory 往返，极大地提升了执行效率。

| 并行部分时间（ms） | 整体运行时间（ms） | 端到端运行时间（ms） |
|---|---|---|
| 0.1748 | 85.562 | 207.819 |
| 0.1894 | 81.787 | 212.653 |
| 0.1772 | 88.043 | 208.276 |

实验数据显示，通过引入“算子融合”与“寄存器缓存”策略，GPU 核心计算时间从平均 0.23ms 稳定下降至 0.18ms 左右，获得了约 1.28 倍 的内核加速比

**原因深度解析：** 基础版耗时 0.23ms，其中包含了两次 Kernel 启动开销以及中间特征图的 Global Memory 读写延迟，优化版耗时 0.18ms，虽然单线程的计算负载增加了（每个线程计算 4 次卷积），但由于数据直接在寄存器中流转，完全削减了对 Global Memory 的中间写入和读取，这证实了实验前的假设：对于 5×5 小卷积核任务，显存带宽是比计算算力更紧缺的资源，减少访存比减少计算更能带来性能提升。

## 总结

本次实验基于 NVIDIA CUDA 异构计算平台，深入研究了 BMP 图像高斯卷积与池化算法的大规模并行实现。通过从基础版到优化版的迭代开发，不仅掌握了 CUDA 编程模型，更深刻理解了 GPU 性能优化的核心逻辑。

a) **基础实现：**成功利用 CUDA Grid-Block-Thread 模型替代 CPU 循环，实现了图像处理的细粒度并行，在 Kernel 层面，GPU 将计算耗时从 CPU 的百毫秒级压缩至 0.2ms 级别，展现强大算力吞吐。

b) **进阶优化：**在 CPU 编程经验中（如 Pthread 实验），我们往往认为“减少乘法次数（降低复杂度）”就是优化，在 GPU 编程中，显存带宽往往比计算能力更昂贵，算子融合虽然增加了单个线程的计算负载，但因为它切断了中间数据的显存通路，直接在芯片内部（寄存器）消化数据，反而取得了最佳性能。

c) **心得体会：**在现代 GPU 开发中，计算换显存是一种极其重要的优化策略。